



ANÁLISE E MODELAGEM PREDITIVA

Orlando da Silva Junior

ANÁLISE E MODELAGEM PREDITIVA

1ª edição

Londrina
Platos Soluções Educacionais S.A
2021

© 2021 por Platos Soluções Educacionais S.A.

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, da Platos Soluções Educacionais S.A.

Diretor Presidente Platos Soluções Educacionais S.A

Paulo de Tarso Pires de Moraes

Conselho Acadêmico

Carlos Roberto Pagani Junior
Camila Turchetti Bacan Gabiatti
Camila Braga de Oliveira Higa
Giani Vendramel de Oliveira
Gislaine Denisale Ferreira
Henrique Salustiano Silva
Mariana Gerardi Mello
Nirse Ruscheinsky Breternitz
Priscila Pereira Silva
Tayra Carolina Nascimento Aleixo

Coordenador

Henrique Salustiano Silva

Revisor

Amanda Souza da Silva

Editorial

Alessandra Cristina Fahl
Beatriz Meloni Montefusco
Carolina Yaly
Mariana de Campos Barroso
Paola Andressa Machado Leal

Dados Internacionais de Catalogação na Publicação (CIP)

Silva Junior, Orlando da
S586a Análise e modelagem preditiva / Orlando da Silva Junior,
São Paulo: Platos Soluções Educacionais S.A., 2021.
44 p.

ISBN 978-65-89881-06-3

1. Modelagem. 2. Preditiva. 3. Dados. I. Título.

CDD 658.1

Evelyn Moraes – CRB 010289/O

2021
Platos Soluções Educacionais S.A
Alameda Santos, nº 960 – Cerqueira César
CEP: 01418-002 — São Paulo — SP
Homepage: <https://www.platosedu.com.br/>

ANÁLISE E MODELAGEM PREDITIVA

SUMÁRIO

Engenharia de Atributos _____	05
Avaliação de Modelos Preditivos _____	18
Otimização de modelos _____	30
Aplicações de análise e modelagem preditiva _____	42

Engenharia de Atributos

Autoria: Orlando da Silva Junior

Leitura crítica: Amanda Souza da Silva



Objetivos

- Explorar problemas de análise e modelagem preditiva.
- Descrever cenários para uso de engenharia de atributos.
- Aplicar técnicas de engenharia de atributos.

1. Introdução à Engenharia de Atributos

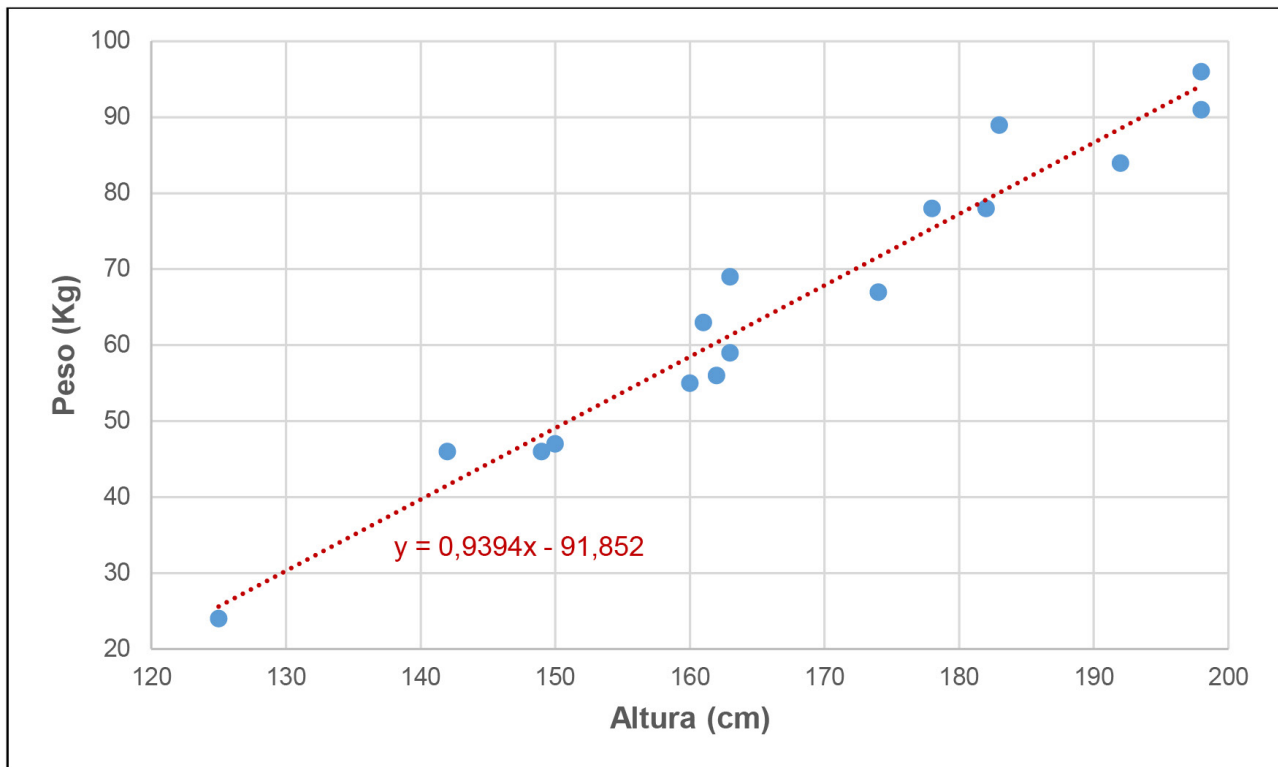
A análise preditiva (ou ***Analytics***) é uma disciplina focada em entender os porquês de um determinado fenômeno estar acontecendo e/ou o que vai acontecer com esse fenômeno no futuro. Para realizar um estudo que busque descrever uma situação atual ou preveja o que acontecerá em um futuro próximo, devemos construir um modelo que seja o mais próximo possível da realidade que queremos observar.

Como usamos estatística em *Analytics*, o nosso modelo será uma forma matemática simplificada de realizar aquela aproximação com o futuro. E, para isso, utilizaremos funções e equações que relacionem a realidade com a nossa observação limitada.

Por exemplo, suponha que você precise reportar o peso de uma grande quantidade de bananas. Em vez de você medir o peso de todas as bananas, já que isso pode levar muito tempo, você pode selecionar 30 bananas que representem a maioria delas, calcular o peso de cada uma e computar ao final o peso médio e o desvio padrão. Reportando apenas esses dois números (a média e o desvio padrão), você terá construído um modelo que represente o conjunto maior de bananas.

Outro exemplo é tentar representar o peso (em Kg) de uma pessoa de acordo com a sua altura (em cm) por meio de uma linha reta após um experimento conduzido com 16 pessoas que tiveram o peso e a altura mensurados. A Figura 1 mostra, graficamente, essa relação na linha vermelha traçada de maneira aproximada entre os 16 pontos azuis. A equação destacada em vermelho $y = 0,9394x - 91,852$ descreve o modelo encontrado para este exemplo.

Gráfico 1 – Gráfico Altura *versus* Peso



Fonte: elaborado pelo autor.

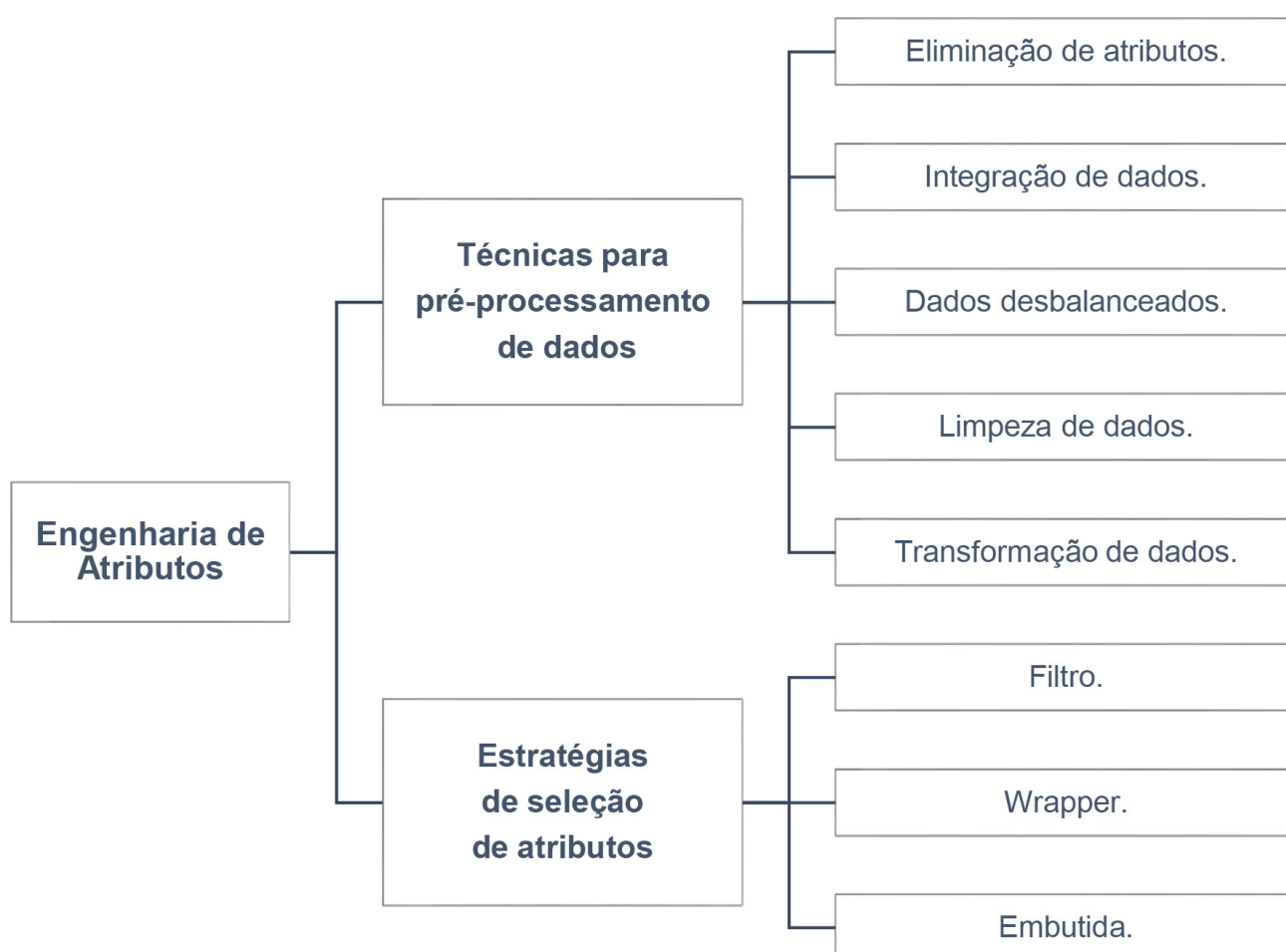
Em problemas mais complexos, com uma maior quantidade de dados e variáveis (atributos), nem sempre encontraremos a situação perfeita. Por exemplo, podemos detalhar o estudo anterior para verificar a relação altura *versus* peso observando o sexo da pessoa. Porém, como quantificar a variável sexo ("M" ou "F"), que não é uma variável numérica, para incluí-la no modelo? Sabemos que essa é uma variável importante nesse tipo de problema, já que homens e mulheres apresentam características físicas distintas.

Nessa situação e em muitas outras, a **Engenharia de Atributos** nos ajudará na condução de problemas complexos e que exijam uma atenção maior na quantidade e qualidade das variáveis e, também, dos dados. Ela pode ser considerada uma etapa da análise preditiva e destaca-se como uma das etapas mais importantes para o processo de modelagem, transformando dados brutos em atributos úteis.

Em problemas reais, acredita-se que até 80% dos esforços sejam consumidos nesta fase durante o desenvolvimento de aplicações de *Machine Learning*. Segundo Pyle (1999), 60% dos esforços na preparação dos dados correspondem a 15% na taxa de sucesso do projeto.

Para entender como a Engenharia de Atributos trabalha em Analytics, nas próximas seções serão apresentadas as duas principais áreas envolvidas nesse processo, que são as técnicas para pré-processamento de dados e as estratégias para seleção de atributos. Um esquema resumido dos tópicos abordados nessas áreas é apresentado na Figura 1.

Figura 1 – Esquema-resumo da Engenharia de Atributos



Fonte: elaborada pelo autor.

2. Técnicas para pré-processamento de dados

Quando falamos em pré-processamento de dados, nos referimos a técnicas que podem incluir, deletar ou transformar os dados contidos em uma base de dados. Embora muitas vezes tenhamos o foco nos processos de modelagem e *Machine Learning* em *Analytics*, a preparação de dados pode fazer a diferença no desempenho preditivo do modelo, além de, em muitas situações, ser necessária para a aplicação de algoritmos.

Nas seções seguintes encontraremos as principais técnicas para pré-processamento de dados. Ao se deparar com um problema analítico, você deverá selecionar uma ou mais técnicas para resolver problemas específicos no conjunto de dados, atributo ou objeto (exemplos ou amostras).

2.1 Eliminação de atributos

Eliminar atributos manualmente é uma tarefa necessária quando nos deparamos com atributos que podem não colaborar com a solução do problema que estamos estudando. A técnica é bastante simples: basta remover do conjunto de dados o(s) atributo(s) que estão atrapalhando. Porém, atenção: antes de remover valores, avalie se a remoção pode prejudicar o restante do banco de dados.

Para exemplificar, a partir de Mitchell (1997), suponha construir um modelo que informe, automaticamente, se uma pessoa deve ou não jogar futebol no final de semana, considerando as condições climáticas. Assim, durante 14 semanas observam-se as condições climáticas e as anotam em uma planilha, junto com a sua opinião (“Sim” ou “Não”) sobre jogar futebol. O Quadro 1 sintetiza esse levantamento de dados.

Quadro 1 – Conjunto com dados climáticos

Semana	Região	Tempo	Temperatura	Umidade	Vento	Partida
1	Norte	Ensolarado	38,9	86	Fraco	Não
2	Norte	Ensolarado	29,4	65	Forte	Não
3	Norte	Nublado	25,4	90	Fraco	Sim
4	Norte	Chuvoso	22,5	75	Fraco	Sim
5	Norte	Chuvoso	19,3	96	Fraco	Sim
6	Norte	Chuvoso	21,5	80	Forte	Não
7	Norte	Nublado	20,7	70	Forte	Sim
8	Norte	Chuvoso	23,4	85	Fraco	Sim
9	Norte	Ensolarado	24,9	90	Forte	Sim
10	Norte	Nublado	22,7	95	Forte	Sim

Fonte: elaborado pelo autor.

Observe no Quadro 1 que os atributos Semana e Região são irrelevantes para o problema em questão, pois:

- O atributo Semana é apenas um identificador do registro correspondente aos valores coletados naquela semana. Nenhum dos valores (1 a 10) poderá influenciar na decisão de jogar futebol.
- O atributo Região repete-se em todos os registros e, neste caso, ele se torna irrelevante por não conter informação que ajude a discriminar os objetos do conjunto de dados.

Com os atributos Semana e Região removidos manualmente, agora será possível, a partir da 11ª semana, com o modelo pronto em mãos, fornecer os valores apenas para os atributos de entrada Tempo, Temperatura, Umidade e Vento, e deixar que o modelo responda no atributo de saída Partida se uma pessoa deverá ou não sair de casa para jogar futebol.

2.2 Integração de dados

A integração de dados é uma tarefa que consiste na manipulação de mais de um conjunto de dados (FACELLI et al., 2011). Muitas vezes, os dados que precisamos estão distribuídos em diversos servidores, dispositivos ou aplicações, sendo necessário identificá-los e combiná-los.

Como o atributo Semana do exemplo anterior pode também se tratar de um identificador, não devemos removê-lo antes de localizá-lo nos conjuntos de dados que farão a integração com o conjunto de dados original.

2.3 Dados desbalanceados

Quando falamos sobre dados desbalanceados, estamos nos referindo ao desequilíbrio existente na quantidade de objetos entre as classes de um problema preditivo. Se esse desequilíbrio não for tratado antes da modelagem, o desempenho do modelo poderá ser afetado.

Por exemplo, no problema da partida de jogos de futebol, observe que o atributo de saída (ou atributo-alvo) Partida possui apenas três registros como “Não” (30%), enquanto existem sete como “Sim” (70%). Esse desbalanceamento pode induzir o modelo a um viés que o faça prevalecer respostas positivas a novas situações.

Para corrigir esse problema, a melhor estratégia é gerar novos dados a partir do processo original e equilibrar as classes. Porém, na maior parte das vezes, isso não é possível, e uma alternativa é balancear os dados de maneira artificial (FACELLI et al., 2011). Assim, podemos tanto incluir novos objetos à classe minoritária quanto eliminar objetos da classe majoritária.

2.4 Limpeza de dados

O processo de limpeza de dados deve ser feito observando-se a qualidade dos dados, garantindo a sua adequação ao uso pretendido. Entre os principais problemas com a qualidade, encontramos:

- **Dados ruidosos:** possuem erros ou valores diferentes do esperado. Por exemplo, "Idade = 109".
- **Dados inconsistentes:** contradizem valores de outros atributos do mesmo objeto. Por exemplo, "Cidade = São Paulo" e "UF = RJ".
- **Dados redundantes:** ocorre quando dois ou mais objetos possuem os mesmos valores para todos os atributos. Por exemplo, "Autor = Maria Clara Ribeiro" e "Escritor = Maria Clara Ribeiro".
- **Dados incompletos:** ausência de valores para alguns dos atributos em parte dos dados. Por exemplo, "Nome = João Paulo Oliveira" e "Idade =" (atributo sem valor).

Para cada um desses problemas, você encontrará diversas técnicas para tratá-los.

Para o **tratamento de dados ruidosos**, é preciso considerar os dados que aparentemente não pertencem à distribuição estatística original. Um indicador da possível presença de ruído são os *outliers*, que consistem em valores considerados como exceções raramente encontradas.

Para tratar *outliers*, você poderá usar técnicas de agrupamento a fim de identificar o ruído nos valores que não conseguiram se agrupar (DE CASTRO; FERRARI, 2016). Ao encontrá-lo, basta removê-lo manualmente.

Para o **tratamento de dados inconsistentes**, deve-se retornar à etapa de integração de dados e verificar em que momento a integridade foi violada e, então, realizar a correção.

No **tratamento de dados redundantes**, você observará inicialmente se a redundância existe no objeto ou no atributo. Podem existir tanto objetos redundantes quanto atributos com valores redundantes. Um atributo é redundante se o seu valor para todos os objetos pode ser deduzido a partir do valor de um ou mais atributos. Por exemplo, Idade e Data de Nascimento, que apresentam as mesmas informações preditivas. No caso de objetos redundantes, pode-se apenas fazer a identificação dos objetos e a eliminação das redundâncias.

Como alternativas para o **tratamento de dados incompletos**, considere a eliminação de objetos com valores ausentes, o preenchimento manual do valor faltante ou a aplicação de algum algoritmo que estime o valor do atributo por meio de um processo de inferência.

Essas e muitas outras técnicas podem ser aplicadas para a melhoria da qualidade de dados. Consulte as referências desta unidade para saber mais.

2.5 Transformação de dados

A transformação de dados é uma das etapas mais importantes para a modelagem preditiva. Sem a transformação prévia dos dados à modelagem, os modelos desejados poderão ser gerados com defeitos, erros estatísticos ou nem serem construídos, a depender do método e do algoritmo de *Machine Learning* selecionado.

Redes neurais, por exemplo, trabalham apenas com dados numéricos. Ao tentar utilizar valores simbólicos, nenhum modelo poderá ser construído. Assim, quando o conjunto de dados apresenta atributos simbólicos e essa técnica deverá ser utilizado, os valores desses atributos deverão ser convertidos para valores numéricos (FACELLI et al., 2011).

Dos tipos de transformação existentes, considere:

- A transformação de um valor simbólico nominal para numérico.
- A transformação de um valor simbólico ordinal para numérico.
- A transformação de um valor numérico para simbólico.

Na **transformação de um valor simbólico nominal para numérico** e na **transformação de um valor simbólico ordinal para numérico**, devemos observar uma das seguintes situações:

1. Se o atributo é ordinal e binário (possui apenas dois valores distintos) ou indica a ausência/presença da característica, podemos simplesmente utilizar os numerais 0 e 1.
2. Se o atributo é ordinal e possui múltiplos valores, a conversão se estenderá além dos numerais binários. Para essa transformação, a ordem e a distância entre os valores deverão ser preservadas (FACELLI et al., 2011).
3. Se o atributo é nominal e possui múltiplos valores, podemos usar a codificação 1-de-n, em que cada sequência terá apenas uma informação codificada com valor 1 e as demais com valor 0 (DE CASTRO; FERRARI, 2016). O Quadro 2 mostra como essa transformação pode ser realizada para o atributo Tempo, que possui três valores possíveis distintos (“Ensolarado”, “Nublado” e “Chuvoso”).

Quadro 2 – Transformação de atributo nominal multivalorado

Tempo	Ensolarado	Nublado	Chuvoso
Ensolarado	1	0	0
Ensolarado	1	0	0
Nublado	0	1	0
Chuvoso	0	0	1
Chuvoso	0	0	1
Chuvoso	0	0	1
Nublado	0	1	0

Chuvoso	0	0	1
Ensolarado	1	0	0
Nublado	0	1	0

Fonte: elaborado pelo autor.

Para a **transformação de um valor numérico para simbólico**, também observaremos se o atributo é binário. Se o atributo tiver apenas dois valores, podemos realizar a transformação binária e associar um nome a cada um dos dois valores. Se o atributo for multivalorado e não existir uma sequência ordenada entre os valores, cada valor poderá ser substituído por um nome ou uma categoria. Esse processo é também conhecido como **discretização**.

Durante o processo transformação de dados dos atributos, chegaremos mais próximos do processo de aprendizagem e na escolha dos algoritmos de *Machine Learning* para o nosso problema.

3. Seleção de atributos

A seleção de atributos é uma técnica da Engenharia de Atributos que visa selecionar novos atributos, formando um subconjunto dos atributos antigos, para reduzir a dimensão do conjunto de dados original.

Embora muitos algoritmos de *Machine Learning* sejam projetados para selecionar os atributos mais apropriados de acordo com o critério de representação do próprio algoritmo, um conjunto de técnicas externas pode colaborar na escolha das melhores características.

Quando a seleção de atributos é realizada automaticamente, podemos aplicar uma das três abordagens principais (CAI et al., 2018):

- **Filtro:** a seleção de atributos acontece antes da geração do modelo de aprendizado e o objetivo é a filtragem dos atributos menos importantes, conforme algum critério pré-determinado.

- **Wrapper:** a seleção de atributos também acontece paralelamente à utilização do algoritmo. A cada iteração, um subconjunto de dados é selecionado para o algoritmo de aprendizagem. O processo é repetido para cada subconjunto de atributos até que um determinado critério de parada seja satisfeito.
- **Embutida:** utiliza um critério interno do algoritmo para realizar a seleção de atributos.

Embora a seleção automática seja menos trabalhosa, o processo manual é a melhor forma de compreendermos o problema de *Analytics*. Porém, quando os dados são não-estruturados (texto, imagens, áudios), a seleção manual pode não explicar adequadamente os atributos selecionados para o algoritmo de aprendizagem.

Entre as principais vantagens da seleção de atributos, estão:

- A simplificação do modelo preditivo.
- A redução do custo computacional na construção do modelo.
- O aumento no desempenho preditivo dos algoritmos de *Machine Learning*, quando utilizados.
- Uma melhor interpretabilidade dos resultados de desempenho, uma vez que o estudo prévio dos atributos foi realizado.

Nesta unidade, você conheceu cenários que podem se repetir ou se assemelhar em problemas do mundo real. Tanto as técnicas de pré-processamento de dados quanto as abordagens para a seleção automática de atributos podem ser usadas em qualquer contexto, desde que o problema estudado exija a aplicação das estratégias escolhidas por você. O conhecimento dessas principais técnicas poderá ajudá-lo a tomar melhores decisões em problemas de análise e modelagem de dados.



Referências

CAI, Jie et al. Feature selection in machine learning: A new perspective.

Neurocomputing, Amsterdã, v. 300, p. 70-79, 2018. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0925231218302911>. Acesso em: 11 maio 2021.

DE CASTRO, Leandro N.; FERRARI, Daniel G. **Introdução à mineração de dados: conceitos básicos, algoritmos e aplicações**. São Paulo: Saraiva, 2016.

FACELLI, Katti et al. **Inteligência Artificial: uma abordagem de aprendizado de máquina**. Rio de Janeiro: LTC, 2011.

MITCHELL, Tom M. **Machine learning**. New York: McGraw-Hill, 1997.

PYLE, Dorian. **Data Preparation for Data Mining**. São Francisco: Morgan Kaufmann Publishers, 1999.

Avaliação de Modelos Preditivos

Autoria: Orlando da Silva Junior

Leitura crítica: Amanda Souza da Silva



Objetivos

- Compreender o processo de avaliação de desempenho de modelos preditivos.
- Estudar métricas e medidas de desempenho de modelos preditivos.
- Aplicar técnicas de reamostragem em problemas de classificação de dados.
- Avaliar o desempenho de modelos preditivos.



1. Introdução à avaliação de modelos preditivos

Os recentes esforços empregados pelas grandes empresas em adotar o *Machine Learning*, têm provocado muito interesse nos profissionais de tecnologia e de muitas outras áreas. A curiosidade a respeito do funcionamento de algoritmos como SVMs e da construção de redes neurais usando TensorFlow, têm, muitas vezes, deixado de lado a importância em avaliar corretamente as saídas geradas por essas técnicas.

Atualmente, apesar da disponibilidade quase infinita de recursos, como a infraestrutura tecnológica de data centers, o volume “terabático” de milhares de bases de dados e algoritmos sofisticados de aprendizagem cognitiva, não devemos deixar de lado a importância de uma avaliação rigorosa até mesmo dos modelos preditivos mais simples.

A seguir, será apresentado um modelo genérico de processo para a avaliação de modelos preditivos. Esse modelo, poderá ser usado em problemas de classificação e regressão.

Nas próximas seções, você estudará técnicas para avaliar o desempenho de modelos preditivos. Ao longo do estudo, você conhecerá estratégias para o tratamento amostral de dados e as principais métricas de avaliação em problemas de Analytics.

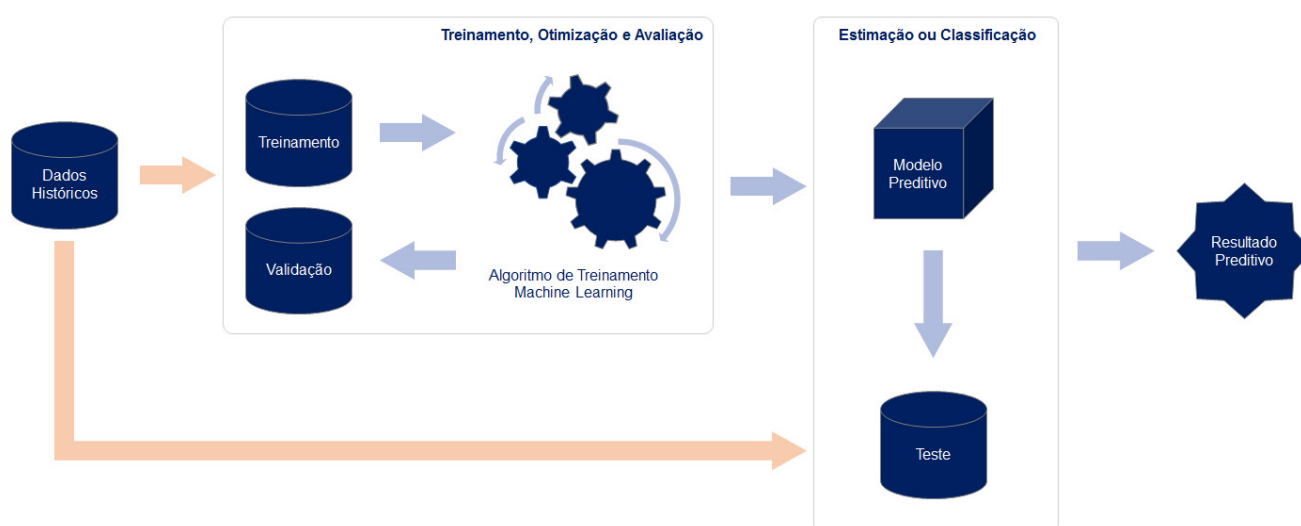
1.1 Processo de avaliação

Como podemos observar, a existência de diferentes algoritmos de *Machine Learning* deixa clara a visão de que não existe uma única técnica capaz de resolver todos os problemas. Experimentar essas técnicas, com suas diferentes variações e hiperparâmetros, é também um trabalho importante a ser realizado para o alcance do desempenho ótimo.

Assim, a experimentação em *Machine Learning* requer o desenho de um processo capaz de avaliar de maneira justa os algoritmos que estão sendo colocados à prova.

A Figura 1 ilustra um modelo genérico para o processo de avaliação de desempenho de algoritmos preditivos. Esse modelo é formado por três etapas: a divisão da base de dados histórica; o treinamento, otimização e avaliação do algoritmo nos dados; e a predição do modelo gerado em novos dados, obtendo o resultado preditivo.

Figura 1 – Modelo genérico de processo para avaliação de modelos preditivos



Fonte: elaborada pelo autor.

Na primeira etapa, os dados históricos são divididos em três bases de dados distintas. Essa separação acontece porque, ao empregar os mesmos dados para treinamento e predição, o desempenho preditivo computará estimativas otimistas, uma vez que todos os algoritmos de *Machine Learning* tentam melhorar de alguma forma o seu desempenho preditivo nesses dados durante a fase de treinamento (FACELLI et al., 2011). Dessa forma, devemos definir subconjuntos disjuntos de treinamento, validação e teste, sendo que:

- **Dados de treinamento:** compreendem a maior fatia dos dados históricos e correspondem aos dados que serão fornecidos como entrada para o algoritmo de aprendizagem.
- **Dados de validação:** realizarão os ajustes no modelo, otimizando o modelo e trazendo melhores resultados. Em geral, são particionados dos dados de treinamento.
- **Dados de teste:** compreender a menor fatia e simulam a apresentação de novos dados nunca apresentados ao preditor.

Na segunda etapa, os dados de treinamento serão utilizados pelo algoritmo de aprendizagem para a construção do modelo de decisão. Esse modelo será constantemente aperfeiçoado por meio da avaliação realizada durante o aprendizado nos dados de validação. Em algoritmos de *Machine Learning* tradicionais, o treinamento é encerrado assim que todos os dados forem processados.

Por fim, na terceira etapa, o melhor modelo preditivo encontrado na etapa anterior deverá ser aplicado nos dados de teste para que possamos encontrar o desempenho preditivo do modelo, observando alguma medida de desempenho. A escolha dessa métrica dependerá de muitos fatores, como o tipo de problema que está sendo resolvido e a qualidade dos dados disponíveis.

Nas seções seguintes, você conhecerá algumas dessas métricas estudando diferentes problemas de *Analytics*.

1.2 Métricas de Erro

Ao avaliarmos o desempenho de um algoritmo aplicado para problemas de regressão ou classificação, queremos analisar o desempenho da função preditora construída por esse algoritmo. Essa função será utilizada no processo de rotulação dos novos dados ainda não vistos

pelo algoritmo e, por isso, precisamos saber o quão eficiente ela será na prática. E, para descobrirmos essa eficiência, computaremos algumas métricas de erro sobre essa função.

Durante nossos experimentos, as métricas de erro deverão ser computadas sobre o subconjunto dos dados de teste. Após a implantação do modelo ajustado, as métricas poderão ser computadas sobre os novos dados nunca vistos pelo preditor.

Embora problemas de classificação e regressão sejam tratados como problemas supervisionados, o rótulo de cada tipo de problema exige que trabalhemos com métricas diferentes. Enquanto na classificação os rótulos são variáveis com valores nominais, na regressão as variáveis trabalham valores numéricos.

Em problemas de classificação, usaremos a **taxa de erro**. Assim, vamos considerar que a taxa de predições incorretas de um classificador é a taxa de erro desse preditor. E que a avaliação desse classificador será realizada para um conjunto de dados com objetos, sendo a taxa de erro correspondente à seguinte equação (FACELLI et al., 2011):

$$erro(\hat{f}) = \frac{\sum_{i=1}^n I(y_i \neq \hat{f}(x_i))}{n}$$

Nessa equação, construímos a função para dizer que o classificador $\hat{f}$ acertou a predição quando $y_i = \hat{f}(x_i)$, e errou a predição quando $y_i \neq \hat{f}(x_i)$. Dessa forma, contaremos quando o classificador acertar, ou seja, quando o resultado for verdadeiro. Caso o classificador erre, contaremos +0, não contabilizando nenhuma pontuação. A taxa de erro deve variar entre 0 e 1, sendo os valores mais próximos de 0 melhores para a avaliação do modelo preditivo.

Também podemos computar a taxa de acerto (ou acurácia) como sendo o complemento da taxa de erro. Para a **acurácia**, quanto mais próximo

de 1 estiver o resultado do cálculo, melhor será o desempenho preditivo do classificador. Podemos computar a acurácia pela seguinte equação:

$$acurácia(\hat{f}) = 1 - erro(\hat{f})$$

No caso da regressão, usaremos principalmente o **erro quadrático médio**, considerando um regressor $\hat{f}$. O erro desse regressor é calculado por meio da distância entre o valor real y_i e o valor predito por $\hat{f}(x_i)$, sendo sempre positivo e idealmente com valores baixos. Podemos calcular o erro quadrático médio pela seguinte equação:

$$MSE(\hat{f}) = \frac{\sum_{i=1}^n (y_i - \hat{f}(x_i))^2}{n}$$

Existem muitas outras métricas de erro para a avaliação de problemas de classificação e regressão. As métricas que você conheceu até aqui são suficientes para a aplicação na maior parte dos problemas preditivos. Outras métricas poderão ser aplicadas em contextos específicos do problema que você estiver resolvendo.

2. Medidas de Desempenho para classificação

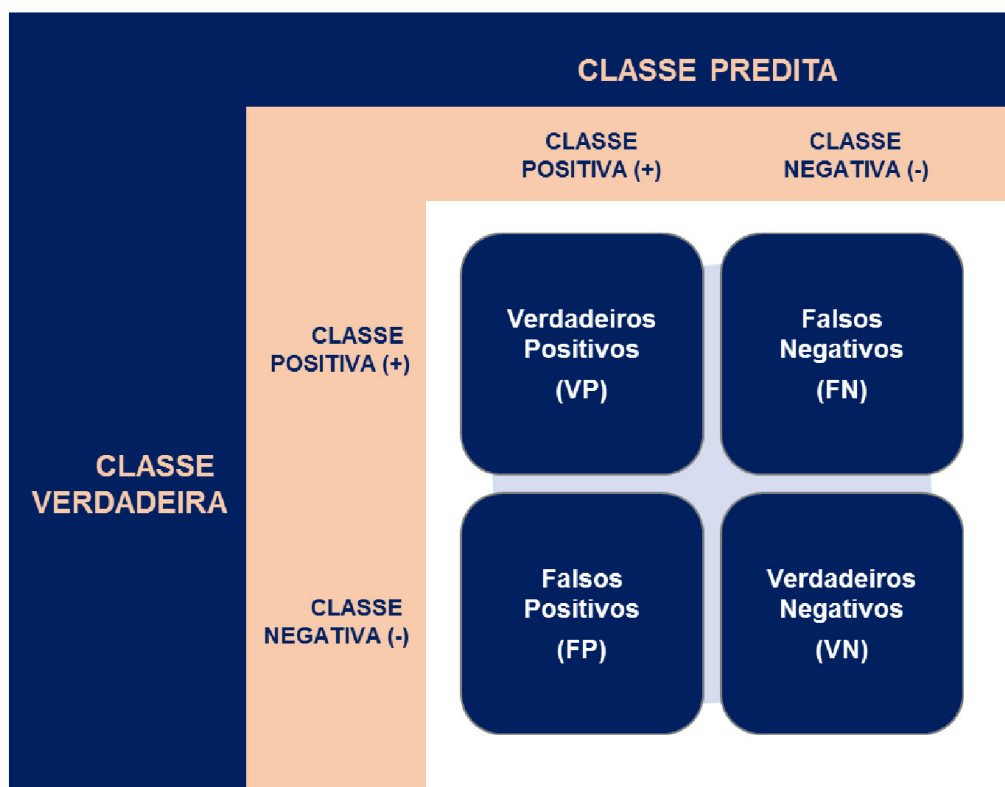
Uma alternativa muito utilizada às métricas de erro apresentadas anteriormente para problemas de classificação é a matriz de confusão, que apresenta a quantidade de predições corretas e incorretas de cada classe.

Uma **matriz de confusão** para um problema com duas classes é apresentada na Figura 2. Consideramos inicialmente as classes com os termos “classe positiva (+)” e “classe negativa (-)” para designar as classes que estão sendo avaliadas. Em seguida, atribuímos os valores

computados da medição ao quadrado correspondente da matriz conforme a seguinte regra:

- VP corresponde aos **verdadeiros positivos**, ou seja, à quantidade de objetos positivos corretamente classificados como positivos pelo preditor.
- FN corresponde aos **falsos negativos**, ou seja, à quantidade de objetos positivos incorretamente classificados como negativos.
- FP corresponde aos **falsos positivos**, ou seja, à quantidade de objetos negativos incorretamente classificados como positivos.
- VN corresponde aos **verdadeiros negativos**, ou seja, à quantidade de objetos negativos corretamente classificados como negativos pelo preditor.

Figura 2 – Matriz de confusão para um problema binário



Fonte: elaborada pelo autor.

Para compreender melhor a ideia de classe positiva e negativa, imagine o seguinte cenário: um banco precisa conceder crédito a seus clientes com base em suas informações de consumo. Como existem muitas variáveis para analisar, o banco decide utilizar um modelo de *Machine Learning* para separar os clientes que tem maior probabilidade de pagar o empréstimo daqueles com baixa probabilidade. Assim, o banco pode dizer que os “bons pagadores” fazem parte da classe positiva, enquanto os “maus pagadores” compõem a classe negativa.

Em problemas com mais de duas classes, a matriz de confusão também poderá ser construída, bastando observar que a matriz deverá ter a dimensão $k \times k$, sendo k a quantidade de classes avaliadas. Por exemplo, uma matriz com 10 classes deverá ter 100 valores em uma matriz 10×10 .

Ainda, por meio da matriz de confusão, é possível extrair diversas medidas de desempenho específicas sobre o preditor. Para isso, vamos considerar a quantidade total de objetos $N = VP + FN + FP + VN$. Assim, podemos computar as principais medidas de desempenho (FACELLI et al., 2011):

- **Taxa de erro na classe positiva:** também conhecida como taxa de falsos negativos (TFN).

$$err_+(\hat{f}) = \frac{FN}{VP + FN}$$

- **Taxa de erro na classe negativa:** também conhecida como taxa de falsos positivos (TFP).

$$err_-(\hat{f}) = \frac{FP}{VN + FP}$$

- **Taxa de erro total:** corresponde à soma da diagonal secundária dividida pela quantidade total de objetos avaliados.

$$err(\hat{f}) = \frac{FP + FN}{N}$$

- **Acurácia:** pode ser entendida como um valor que mede o grau de conformidade de um valor obtido com seu valor real.

$$acurácia(\hat{f}) = \frac{VP + VN}{N}$$

- **Precisão:** proporção de objetos positivos preditos corretamente entre todos aqueles preditos como positivos.

$$precisão(\hat{f}) = \frac{VP}{VP + FP}$$

- **Sensibilidade (ou *recall*):** corresponde à taxa de acerto na classe positiva.

$$recall(\hat{f}) = \frac{VP}{VP + FN}$$

- **Especificidade:** corresponde à taxa de acerto na classe negativa.

$$especificidade(\hat{f}) = \frac{VN}{VN + FP}$$

Todas essas medidas trabalham com valores entre 0 e 1. Além disso, tanto a matriz de confusão quanto essas equações podem ser generalizadas para problemas com mais de duas classes. Neste caso, você deverá computar uma medida para cada classe do problema. Por exemplo, um problema com 10 classes distintas deverá ter 10 valores para taxas de erro.



3. Técnicas de reamostragem

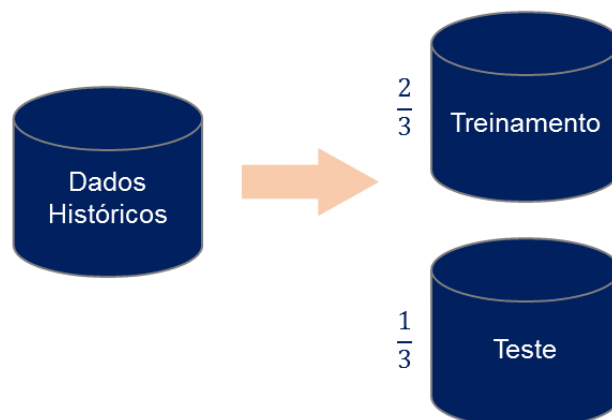
Utilizar o mesmo conjunto de dados para as fases de treinamento e avaliação do preditor pode prejudicar a análise do desempenho do algoritmo, uma vez podemos produzir estimativas otimistas sobre os resultados. Dessa forma, utilizar técnicas de reamostragem é uma estratégia importante para a avaliação de modelos preditivos.

Entre as diversas técnicas de reamostragem, a mais simples é a **amostragem aleatória** (KUHN; JOHNSON, 2013; FACELLI et al., 2011), que consiste em dividir aleatoriamente os dados disponíveis em um conjunto de treinamento a partir do qual o algoritmo de aprendizagem produz $\hat{f}$ e um conjunto de teste em que a precisão de $\hat{f}$ é avaliada.

Quando realizamos essa divisão na proporção p para treinamento e $(1 - p)$ para teste, temos um caso especial comumente chamado de **holdout**. Embora essa técnica seja simples, fácil de ser implementada e bastante usual para grandes conjuntos de dados, esse método tem a desvantagem de não usar todos os dados disponíveis, destinando apenas uma parte à aprendizagem. Esse tipo de separação pode subestimar a taxa de erro, gerando um modelo menos abrangente ou com estimativas mais fracas da precisão real (FACELLI et al., 2011; RUSSELL; NORVIG, 2013).

A Figura 3 ilustra a aplicação da técnica, em que nela dividimos o conjunto de dados em uma proporção de k para treinamento e $1 - k$ para teste. Em geral, usamos $k = \frac{2}{3}$. Por exemplo, se o nosso conjunto de dados possui 300 objetos, usaremos $\frac{2}{3}$ para treinamento (200 objetos) e $\frac{1}{3}$ para teste (100 objetos). Outras proporções não só podem ser utilizadas como muitas vezes devem ser utilizadas durante a experimentação.

Figura 3 – Exemplo de aplicação para a amostragem *holdout*.



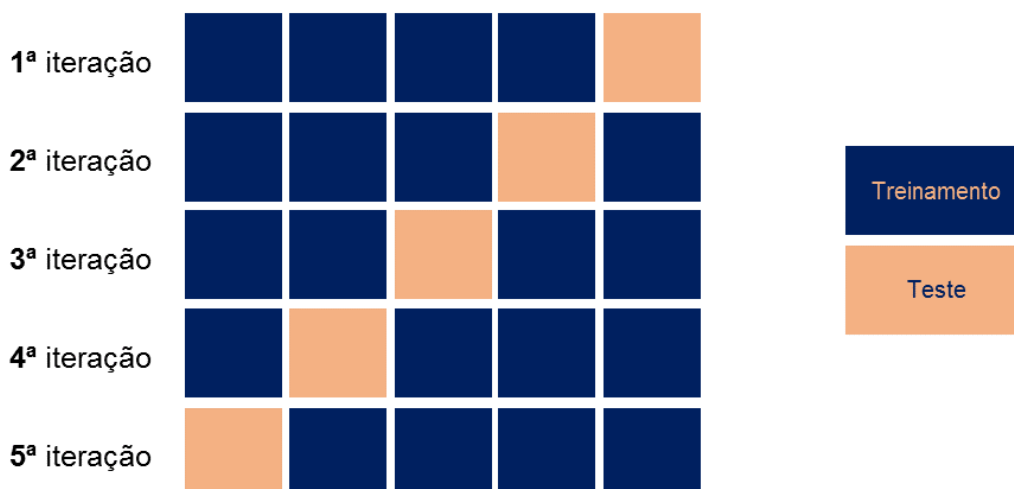
Fonte: elaborada pelo autor.

Por fim, outra técnica bastante aplicada é a **validação cruzada** (FACELLI et al., 2011). Nesse método, o conjunto de dados é dividido em k subconjuntos de tamanho aproximadamente igual. Os objetos dos $k - 1$ subconjuntos são utilizados para treinamento, e o subconjunto restante é utilizado para teste. Esse procedimento é repetido k vezes, sendo que, a cada nova iteração, um subconjunto diferente é utilizado para teste.

A Figura 4 ilustra a aplicação desse método quando $k = 5$. Neste caso, na primeira iteração do método, quatro partições do conjunto de dados serão utilizadas para treinamento e apenas uma será utilizada para teste. Na segunda iteração, os da partição de teste da primeira vez fará parte do conjunto de treinamento, sendo outra partição designada para validar o preditor.

Uma pergunta muito frequente ao utilizar a validação cruzada é: como determinar o valor de k ? Muitos estudos defendem que, embora alguns problemas necessitem de um valor ótimo, a maior parte dos problemas pode adotar $k = 10$ sem maiores preocupações (KUHN; JOHNSON, 2013).

Figura 4 – Validação cruzada com $k = 5$



Fonte: elaborada pelo autor.

Uma das principais vantagens da validação cruzada é que ela pode ser utilizada em conjuntos de dados pequenos ou que não sejam tão grandes a ponto de permitir uma separação única entre conjuntos de treinamento e teste. Em determinados domínios, essa característica é fundamental, pois a ausência de um grande volume de dados não deveria atrapalhar o processo de aprendizagem de máquina. Por outro lado, também é importante notar que, quando $k > 2$, uma parte dos dados passa a ser compartilhada entre os subconjuntos de treinamento, impedindo a completa independência dos subconjuntos.

Nesta unidade, você conheceu a avaliação do desempenho preditivo de algoritmos de *Machine Learning* supervisionado. A partir das técnicas de reamostragem, das métricas de erro e das medidas de desempenho derivadas da matriz de confusão, você poderá concluir na prática e com êxito o processo de avaliação de modelos preditivos.



Referências

FACELI, Katti et al. **Inteligência Artificial:** uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2011.

KUHN, Max; JOHNSON, Kjell. **Applied predictive modeling.** New York: Springer, 2013.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial:** uma abordagem moderna. 3. ed. Rio de Janeiro: Elsevier, 2013.

Otimização de modelos

Autoria: Orlando da Silva Junior

Leitura crítica: Amanda Souza da Silva



Objetivos

- Compreender a importância dos atributos na modelagem preditiva.
- Estudar o processo de indução de modelos.
- Conceituar *overfitting* e *underfitting*.
- Aplicar estratégias para o ajuste de hiperparâmetros.



1. Introdução à otimização de modelos

O desenvolvimento de modelos de *Machine Learning* é uma atividade presente na etapa de modelagem de um projeto de ciência de dados. Porém, não basta construirmos o modelo para que o projeto seja finalizado; precisamos, também, validar os resultados desse modelo, avaliar a sua importância para o problema de negócio que estamos tentando resolver e distribuir os resultados em um formato adequado.

Uma outra atividade que muitas vezes faz parte da modelagem ou posiciona-se ao final do processo, antes da distribuição, é a otimização dos modelos construídos. Otimizar um modelo significa buscar melhorar o desempenho preditivo desse modelo antes de entregá-lo em produção. Podemos até pensar que isso é realizado na fase de modelagem, mas, na maior parte das vezes precisamos deixar essa atividade para o final, a fim de que a metodologia empregada anteriormente possa ser firme o suficiente para que poucos ajustes de otimização sejam necessários.

Outro ponto importante nesse processo é que a nossa primeira melhoria deve ter como base de comparação não o modelo em si mesmo, mas um preditor de base (*baseline*), que é um modelo cujos resultados já temos antes mesmo de começar. Em muitos problemas de classificação binária, o *baseline* é um modelo aleatório que apresenta 50% de acertos, como o ato de jogar uma moeda. Ou seja, se o nosso modelo acertar mais que o *baseline*, precisaremos apenas nos concentrar em ajustá-lo por meio de estratégias de otimização.

Para que você possa compreender o que é importante na otimização do modelo, nesta unidade você vai revisar alguns pontos já estudados, mas com uma visão sobre o processo de construção e otimização de modelos de *Machine Learning*. Ao final, você vai conhecer as principais estratégias utilizadas para o ajuste de hiperparâmetros.

1.1 Atributos

Vamos, inicialmente, pensar no processo de modelagem preditiva como uma “segmentação supervisionada” (PROVOST; FAWCETT, 2016), em que, a partir de uma base de dados, criamos grupos a partir de algo que gostaríamos de prever ou estimar. Nesse sentido, o nosso alvo preditivo pode ser trabalhado de duas formas:

- De maneira positiva, como na descoberta de quais consumidores são mais propensos a comprar um produto.
- De maneira negativa, como na predição de quais clientes são mais suscetíveis de abandonar a empresa quando seus contratos vencerem.

Independentemente do objetivo escolhido, uma ideia fundamental desse processo é a seleção dos atributos que farão parte do modelo. Lembre-se que esses atributos são importantes para o problema e, também, devem ser informativos a respeito das entidades que queremos representar.

Por exemplo, talvez o “tipo sanguíneo” de uma pessoa não seja importante considerando o tipo de produto que ela compra no supermercado. Embora essa seja uma informação adicional a respeito de quem está comprando, o atributo “tipo sanguíneo” em nada nos ajuda no problema, que é a sugestão de produtos para comprar. Em algumas situações, a inclusão de atributos pouco relevantes também pode atrapalhar na solução.

Assim, os atributos escolhidos para a modelagem devem reduzir a incerteza a respeito do que queremos descobrir. Quando temos uma variável alvo, a busca pelos atributos informativos torna-se mais fácil. A nossa busca tem como meta aqueles atributos que apresentam maior correlação com o atributo-alvo. Estatisticamente, a correlação entre os atributos vai nos ajudar a reduzir a incerteza sobre as variáveis.

1.2 Indução de modelos preditivos

Ao construir um modelo preditivo, nosso objetivo é ter uma representação simplificada de uma realidade particular. Os modelos devem servir a um propósito específico, sendo simplificados a partir de um conjunto de pressuposições sobre o que é importante ou não dentro de uma aplicação.

Lembra do tipo sanguíneo na sugestão de produtos? Em análise e modelagem preditiva, um modelo é uma função matemática ou uma declaração lógica utilizada para estimar um valor desconhecido. Embora utilizemos o termo “previsão” (ou predição) muitas vezes, a ideia de estimar um valor parece mais correta quando pensamos que a função pode observar não apenas algo do futuro, como também do presente e do passado. Nos bancos, os modelos de concessão de crédito – para empréstimo e financiamento imobiliário, por exemplo – estimam a probabilidade de um potencial cliente ser inadimplente. Em sistemas de mensagem eletrônica, os modelos para filtragem de SPAM estimam se um novo e-mail é lixo eletrônico.

Quando um novo modelo preditivo é gerado, ele é criado a partir de um processo de indução. Goldstein et al. (2007) definem o que é indução e dedução da filosofia:

- **Indução** é o estudo da obtenção de conclusões que, estritamente falando, vão além das premissas.
- **Dedução** é o estudo das conclusões que podem ser obtidas de premissas.

A partir desses termos filosóficos, entendemos que, para a modelagem preditiva, a indução é generalização de regras gerais a partir de casos específicos. Se você pensar na base de dados como casos específicos de um contexto particular (por exemplo, cadastro individual de alunos de uma escola), o modelo preditivo será a regra geral construída a

partir desses casos por meio de um **algoritmo de indução**, que é o responsável por construir essa regra geral.

Ao pensar na “segmentação supervisionada” como uma intuição para a extração de padrões, podemos imaginar:

1. Um processo de segmentação de uma população em grupos que possuem diferentes valores para o atributo-alvo.
2. Em cada grupo, os exemplos possuem valores diferentes para a atributo-alvo.

Portanto, se a segmentação é realizada com os valores dos atributos que serão conhecidos quando o atributo-alvo for desconhecido, esses segmentos poderão ser utilizados para predizer o valor do atributo-alvo.

Por exemplo, vamos supor que temos um modelo de concessão de crédito. Um segmento desse pode ser expresso na frase “aposentados com mais 67 anos residentes na cidade de São Paulo não são inadimplentes”, em que a indicação de inadimplência representa o atributo-alvo do segmento. O restante da frase define o segmento.

Pensando em como podemos expressar um segmento a partir das suas características, podemos nos perguntar: como podemos decidir se um atributo contém informações importantes sobre o alvo? Essa pergunta é fundamental para a compreensão do modelo de segmentação. Afinal, qual é o segmento que melhor prevê a concessão de crédito?



2. Modelagem preditiva

Na modelagem preditiva, queremos buscar por um modelo do atributo-alvo a partir de outros atributos descritivos. Utilizando dados, produzimos a estrutura de um modelo e os “parâmetros” numéricos do

modelo, que correspondem às estimativas de probabilidade (PROVOST; FAWCETT, 2016).

Uma maneira de aprender um modelo preditivo é partir da especificação da estrutura do modelo, deixando alguns parâmetros numéricos sem especificação. Ao longo da aprendizagem, o processo irá calcular os melhores valores dos parâmetros a partir do conjunto de dados para construir o modelo. Neste caso, o objetivo da mineração de dados é o ajuste dos parâmetros de modo que o modelo se ajuste aos dados da melhor forma possível. Muitos métodos de *Machine Learning* funcionam dessa forma, que é conhecida como modelagem paramétrica.

A **modelagem paramétrica** assume que o conjunto de dados é proveniente de um tipo de distribuição de probabilidade, e faz inferências sobre os parâmetros da distribuição. Em geral, o termo paramétrico refere-se à média e ao desvio-padrão, que são os parâmetros que definem as populações que apresentam distribuição normal.

Em contrapartida, a **modelagem não-paramétrica** corresponde a técnicas que não dependem de nenhuma distribuição de dados particular. Por exemplo, um histograma é uma estimação não-paramétrica de distribuição de probabilidade.

A maior parte dos métodos de *Machine Learning* são paramétricos e baseados em **modelos lineares**. Por exemplo, podemos discriminar (separar) os exemplos de treinamento de duas classes se introduzirmos um limite que é uma linha reta não perpendicular aos eixos.

Com a “segmentação supervisionada” mencionada no início desta unidade, o nosso objetivo é separar os dados em regiões com diferentes valores do atributo-alvo usando uma função matemática. É importante observar que, na maior parte das vezes, nosso modelo terá vários atributos. Ou seja, utilizaremos a **análise multivariada** para fazer

com que o nosso modelo final tenha dois ou mais atributos ao mesmo tempo.

Na prática, a função matemática levará em conta todos (ou a maior parte) dos atributos do nosso problema. Utilizando essas funções, o papel da aprendizagem de máquina é ajustar o modelo para um determinado conjunto de dados. Neste caso, “ajustar” significa que o algoritmo de aprendizagem deverá encontrar um conjunto de pesos da função linear a partir dos atributos x :

$$y = f(x) = w_0 + w_1x_1 + w_2x_2 + \dots$$

Conhecendo os pesos, podemos interpretar a importância de cada atributo a partir do valor associados a eles. Quanto maior o peso, mais importante será o atributo para o problema estudado. Assim, a ideia de **seleção de atributos** relevantes e informativos volta a ser discutida como fundamental para a análise e modelagem preditiva. Aplicaremos essa estratégia a diferentes tarefas, como a classificação e a regressão.



3. *Overfitting e Underfitting*

Nas seções anteriores, foi apresentada, com mais detalhes a importância dos atributos nos problemas de análise preditiva e a relação dos atributos com os modelos. Você também aprendeu a importância em ter um modelo que se ajuste “bem” aos dados de treinamento, e que esse ajuste pode ser controlado por meio de pesos, que são combinados em uma função.

Um problema que queremos estudar a respeito desse ajuste é sobre o quão flexível o nosso modelo deve ser em relação aos dados. Devemos notar aqui que a construção de um modelo tem como finalidade os dados que serão conhecidos após a modelagem, e não os dados de entrada. Dessa forma, lembre-se que sempre buscaremos um modelo que se aplique a novos dados.

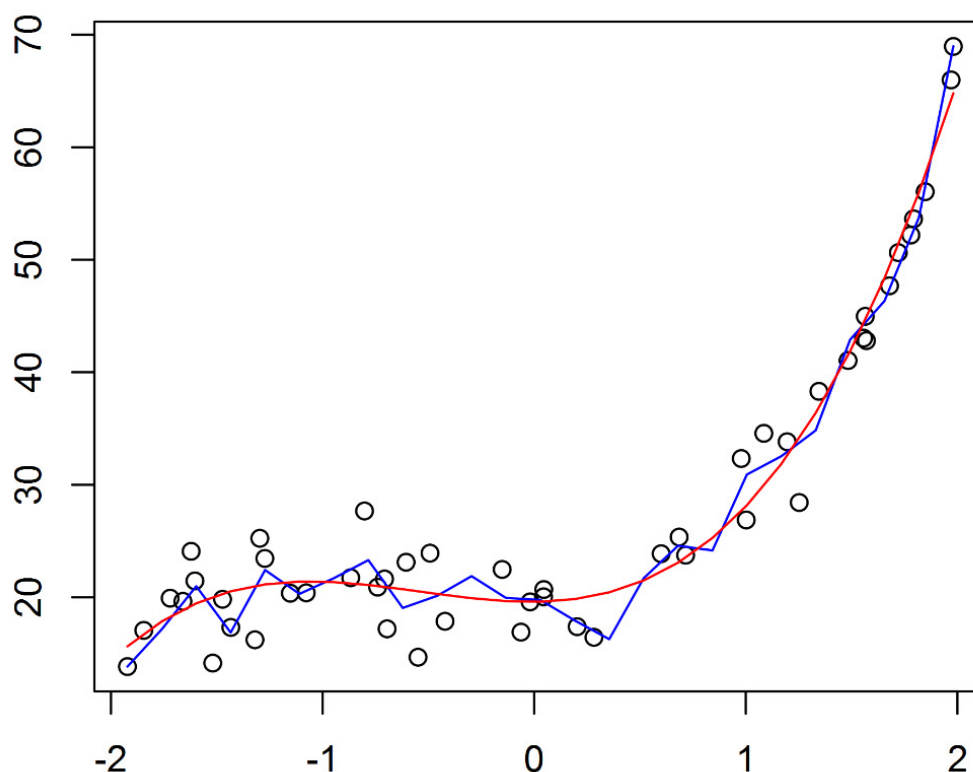
Se, por um lado, a flexibilidade é importante para a busca de padrões, por outro, ela pode reconhecer como padrões o que na verdade são ocorrências ao acaso. Portanto, nosso modelo deve priorizar a **generalização**, ou seja, a predição de novos casos que ainda não foram observados. Sem essa habilidade, o algoritmo de aprendizagem seria responsável apenas por armazenar os dados históricos e consultá-los no momento da predição. Na prática, o modelo não seria “inteligente”.

Sem essa habilidade de generalização, o algoritmo de aprendizagem seria responsável apenas por armazenar os dados históricos e consultá-los no momento da classificação. Por exemplo, em vez de prever se um novo cliente comprará o produto, o algoritmo responde “sim” para um cliente e “não” para todos os outros, memorizando os dados. Esse **modelo de tabela** não é o que estamos buscando para a nossa modelagem preditiva, já que nele não há generalização.

O modelo que não é capaz de extrapolar além do que já foi observado, ajusta-se perfeitamente aos dados de treinamento, que são os dados de entrada. Esse sobreajuste é conhecido como **overfitting**, e é uma característica dos modelos preditivos que apresentam uma tendência em se adaptarem aos dados de treinamento, prejudicando a generalização em pontos de dados não observados.

A Figura 1 apresenta uma comparação entre dois modelos lineares ajustados a um conjunto de pontos. O modelo representado pela linha vermelha, ilustra o modelo ideal para ajuste aos dados, com generalização e flexibilidade. Na linha azul é representado o modelo sobreajustado aos dados. Observe que esse modelo busca pelos pontos de dados e sua curva é menos flexível que a linha do modelo generalizado.

Figura 1 – Comparação entre o modelo ideal (vermelho) e o modelo sobreajustado (azul)



Fonte: elaborada pelo autor.

Ainda que o nosso exemplo do modelo de tabela seja mais didático do que real, é importante observar que o *overfitting* pode existir em qualquer modelo preditivo. Isso não significa, no entanto, que devemos fazer de tudo para que a curva de ajuste deixe de passar em cima de algum ponto de dado. Muitas vezes o nosso problema investigado será tão complexo que um modelo igualmente complexo será necessário, ainda que ele possa nos prejudicar em relação ao sobreajuste. Assim, a melhor estratégia é identificar o *overfitting* e gerenciar a complexidade.

Geralmente, conforme Domingos (1999), os melhores modelos serão aqueles que se encaixam melhor aos dados e que forem mais simples. Na prática, podemos utilizar técnicas de **regularização** para otimizar não apenas o ajuste do modelo aos dados, como também a combinação

de ajuste e complexidade do modelo – dois princípios que colaboram para um bom desempenho preditivo. Por exemplo, em modelos indutivos de áreas de decisão, a poda da árvore é uma estratégia de regularização que reduz o tamanho e a complexidade do modelo, além de reduzir o *overfitting*.

Outro problema sofrido pelos modelos preditivos é o ***underfitting*** (subajuste), que acontece quando o modelo construído faz pouco esforço para se ajustar aos dados. Em geral, modelos com alto viés e baixa variância sofrem de subajuste. Entre as principais soluções para este problema, destacam-se:

- A utilização de mais atributos. Como discutimos anteriormente, a presença de atributos significativos é muito importante para a geração de um modelo preditivo com poucas deficiências. Tentar incluir novos atributos pode ajudar no poder preditivo do modelo.
- A construção de um modelo mais complexo. Construir um modelo mais complexo não quer dizer que o modelo deva ser complexo em sua totalidade. Adicionar complexidade pode melhorar o viés, reduzindo a alta variância.



4. Estratégias de otimização

Um **hiperparâmetro** é um parâmetro que não é diretamente aprendido pelos estimadores. Por exemplo, os parâmetros *C*, *kernel* e *gamma*, e são hiperparâmetros do método SVM. Todos eles são utilizados para controlar a aprendizagem do algoritmo. Ao contrário dos pesos das redes neurais artificiais – que são considerados parâmetros do algoritmo –, os hiperparâmetros não podem ser aprendidos durante o processo de treinamento.

O ajuste (ou otimização) de hiperparâmetros é um problema em *Machine Learning* que trata da escolha dos hiperparâmetros ideais de

um algoritmo de aprendizagem. Essencialmente, segundo Caixeta-Filho (2011), a otimização de hiperparâmetros é um problema matemático cuja solução passa por técnicas específicas.

Além da otimização de hiperparâmetros melhorar o desempenho preditivo do modelo de decisão, podemos controlar o *overfitting* do modelo aos dados de treinamento. Em detalhes, esse processo deve encontrar um conjunto de hiperparâmetros que devolvem um modelo ótimo minimizando uma função de perda em um determinado conjunto de dados independente. A função objetivo que minimiza a função de perda, toma como entrada o conjunto de hiperparâmetros e retorna a perda associada. Para encontrar esse modelo ótimo, utilizamos a validação cruzada, na maior parte das vezes.

Existem muitas abordagens para buscar os hiperparâmetros ótimos no espaço de hiperparâmetros, sendo a busca aleatória e a busca em grade as principais estratégias.

Na **busca em grade**, a ideia fundamental é gerar exaustivamente candidatos a partir de uma grade de valores pré-determinados manualmente e varrer o espaço de hiperparâmetros até que os hiperparâmetros ótimos sejam encontrados. Para o algoritmo saber que encontrou o candidato ótimo, ele avalia todas as possibilidades, preferencialmente, no subconjunto de avaliação.

Na **busca aleatória**, a combinação exaustiva da busca em grade é substituída por um conjunto de configurações, em que cada configuração é selecionada a partir de uma distribuição de possíveis valores de parâmetros. Na prática, uma das configurações é selecionada aleatoriamente pelo algoritmo, que a valida sobre o conjunto de dados.

Embora a busca aleatória possa não selecionar o candidato ótimo, ela pode superar a busca em grade quando existe um pequeno conjunto de hiperparâmetros que afeta o desempenho final da aprendizagem

(BERGSTRA; BENGIO, 2012). A busca aleatória também permite que o conhecimento a priori do especialista seja adicionado à busca, tornando-a mais eficiente.

A ideia fundamental dessas estratégias e os princípios da modelagem preditiva que você estudou até aqui compõem o arcabouço de conhecimento para a otimização de modelos preditivos. Nesta unidade, foram abordados a importância dos atributos na modelagem preditiva, o processo de indução de modelos, o *overfitting* e *underfitting*, além da aplicação de estratégias para o ajuste de hiperparâmetros.



Referências

CAIXETA-FILHO, José V. **Pesquisa operacional: técnicas de otimização aplicadas a sistemas agroindustriais**. São Paulo: Atlas, 2011.

BERGSTRA, James; BENGIO, Yoshua. Random search for hyper-parameter optimization. **The Journal of Machine Learning Research**, v. 13, n. 1, p. 281-305, 2012. Disponível em: <https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>. Acesso em: 11 maio 2021.

DOMINGOS, Pedro. The role of Occam's razor in knowledge discovery. **Data mining and knowledge discovery**, v. 3, n. 4, p. 409-425, 1999. Disponível em: <https://link.springer.com/article/10.1023/A:1009868929893>. Acesso em: 11 maio 2021

GOLDSTEIN, Laurence et al. **Lógica: conceitos-chave em filosofia**. Porto Alegre: Artmed, 2007.

PROVOST, Foster; FAWCETT, Tom. **Data science para negócios**. Rio de Janeiro: Alta Books, 2016.

Aplicações de análise e modelagem preditiva

Autoria: Orlando da Silva Junior

Leitura crítica: Amanda Souza da Silva



Objetivos

- Estruturar projetos de ciência de dados.
- Estudar comandos e estruturas de programação especificadas para análise de dados.
- Criar algoritmos para manipulação e modelagem de dados.
- Aplicar conceitos de análise preditiva e classificação em Python.



1. Introdução ao desenvolvimento de projetos de ciência de dados

Conhecer os conceitos de análise e modelagem preditiva são fundamentais para a aplicação em projetos práticos e reais no nosso dia a dia. Estudar uma linguagem de programação com foco em ciência de dados, como R ou Python, também é importante para que você possa ter autonomia na construção de projetos de processamento de dados.

Nesta unidade, estudaremos como estruturar projetos de ciência de dados a partir dos conceitos, definições, métodos e abordagens previamente estudados para análise e modelagem preditiva.

Você também vai estudar os principais comandos e estruturas de programação para analisar e modelar dados. Para isso, escolhemos a linguagem de programação Python em razão da sua facilidade de aprendizagem e atual adoção em massa pelas grandes e médias empresas.

Por fim, você também vai estudar os principais tipos de problemas de análise preditiva e como solucioná-los a partir do estudo da Engenharia de Atributos e otimização de modelos. Para realizar esse estudo, vamos nos concentrar em alguns temas importantes para o seu aprendizado, como:

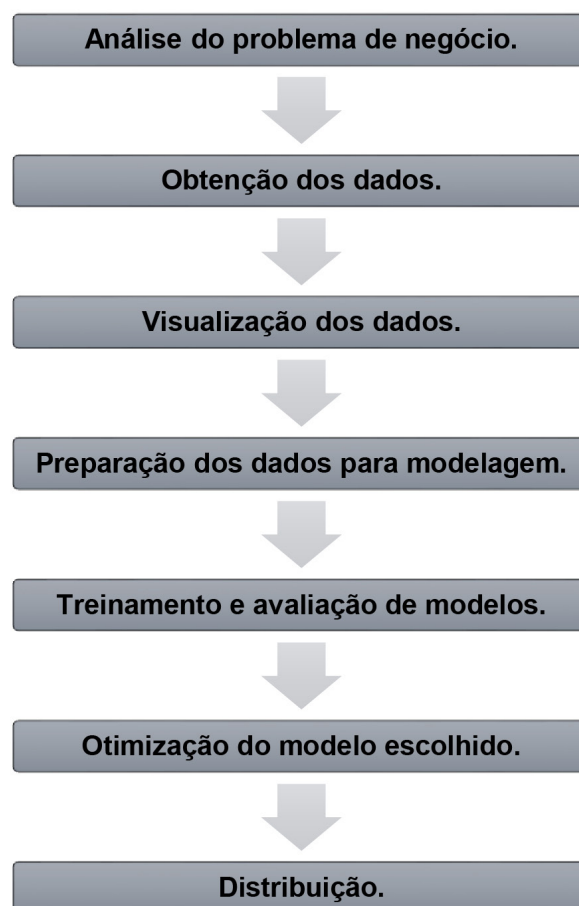
- O processo de desenvolvimento de projetos de ciência de dados.
- A linguagem de programação Python aplicada à análise e dados.
- Mecanismos de programação para modelagem e entrega de modelos e resultados.
- Soluções mais comuns para resolver problemas de modelagem.

- Os principais desafios que podemos encontrar na análise e modelagem preditiva.

1.1 Processo de desenvolvimento de projetos

Assim como no desenvolvimento de software e sistemas, a construção de um projeto de ciência de dados também requer o uso de uma metodologia para que o produto seja construído com qualidade e conforme os requisitos determinados pelo cliente. A Figura 1 ilustra o processo que utilizaremos nesta unidade para construir projetos de ciência de dados usando *Machine Learning*. Esse processo foi inspirado, adaptado e expandido da metodologia CRISP-DM (CHAPMAN et al., 2000), muito popular para **mineração de dados**.

Figura 1 – Processo de ciência de dados para Python



Fonte: elaborada pelo autor.

Na etapa de **análise do problema de negócio**, vamos compreender o problema e o domínio de negócio que estamos tentando resolver. Sem essa compreensão, qualquer tentativa de análise poderá comprometer os resultados da modelagem e fazer com que os esforços empreendidos na codificação se tornem nulos. Nessa etapa, também definiremos o objetivo do nosso projeto, a tarefa que iremos realizar, os métodos de *Machine Learning* que serão aplicados e as métricas que deverão ser coletadas para mensurar os resultados.

A partir da fase de **obtenção dos dados**, trabalharemos diretamente no ambiente de programação, importando as bibliotecas da linguagem e carregando o conjunto de dados em memória.

Após carregar os dados, iremos explorar o conjunto de dados por meio de diagramas, gráficos e tabelas na etapa de **visualização dos dados**. Essa etapa é importante para observarmos como os dados se apresentam a nós e o que podemos extrair deles em um primeiro momento. Ela também é importante para entendermos como os dados deverão ser tratados na próxima etapa.

A fase de **preparação dos dados para modelagem** é destinada ao pré-processamento das linhas e colunas do conjunto de dados estruturado usando as técnicas de Engenharia de Atributos. Essa preparação prévia à modelagem é importante para que os algoritmos selecionados trabalhem corretamente com os dados de entrada.

Com os dados prontos para a modelagem, na etapa de **treinamento e avaliação de modelos** nós aplicaremos os algoritmos de *Machine Learning* selecionados, realizando os experimentos necessários para selecionar o método mais adequado ao problema em análise.

Após a escolha do método, na fase de **otimização do modelo escolhido** trabalharemos com o algoritmo que foi selecionado na etapa anterior com os melhores resultados preditivos. Nesta fase, aplicaremos os

conceitos aprendidos sobre otimização para ajustar os hiperparâmetros do algoritmo e torná-lo ainda mais eficiente.

Por fim, a etapa **distribuição** é destinada à implantação do modelo construído. Como existem diversas formas de distribuirmos o modelo (relatório, planilhas, bases de dados, web, mobile etc.), conheceremos em poucas linhas de código o que é possível fazer nesta etapa.

2. Python para ciência de dados

A linguagem Python é umas principais linguagens utilizadas para ciência de dados. Ela é uma linguagem imperativa e interpretada, muito semelhante a outras linguagens do mesmo tipo, como Pascal, Java, C++ e PHP. Conforme Wazlawick (2018), a linguagem surgiu em 1991 como uma linguagem que procurava simplificar a tarefa de programação.

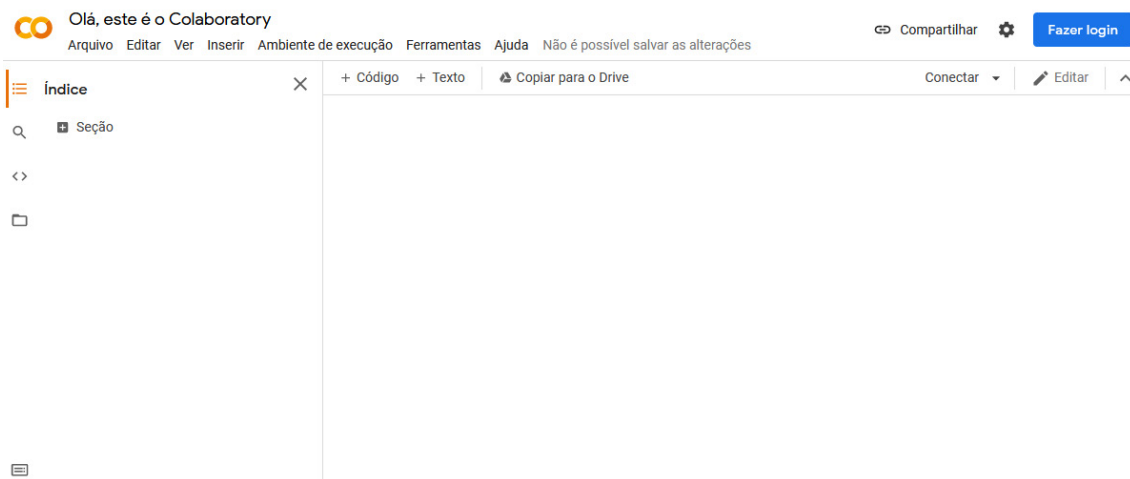
Para *Data Science*, Python desenvolveu uma comunidade grande e ativa de processamento e análise de dados. Segundo Mckinney (2019), por ser uma linguagem robusta para uma engenharia de software de propósito geral, Python se tornou uma linguagem de Computação Científica inovadora para a construção de aplicações de dados.

2.1 Ambiente

Existem muitas opções de ambientes para a programação em Python. A distribuição **Anaconda** é uma delas e já vem preparada com as bibliotecas mais comuns para ciência de dados. Ao contrário do modelo de programação focado em web ou aplicativos móveis, a programação para experimentação tem como objetivo desenvolver códigos para manipular conjuntos de dados. Dessa forma, ter um ambiente que nos permita enxergar claramente as etapas do experimento é primordial.

Tanto Anaconda como o **Google Colab**, que usaremos aqui, estão preparados para a experimentação em ciência de dados. Ambos os ambientes utilizam o modelo de notebooks, como o **Jupyter Notebook**, para a confecção de scripts e a execução de códigos. A Figura 2 apresenta o ambiente de programação online Google Colab.

Figura 2 – Google Colab



Fonte: captura de tela de Google Colab.

2.2 Estruturas de programação

Após conhecermos o ambiente de programação, nosso próximo passo é estudar as principais estruturas da linguagem Python para ciência de dados. Para realizar esse estudo, utilizaremos um conjunto de dados bastante conhecido na comunidade de *Data Science*, que é o conjunto de dados **Titanic**.

A maior parte das aplicações para esse conjunto visa prever os sobreviventes da tragédia em uma tarefa de classificação. Em um problema mais complexo, realizaríamos uma análise do problema de negócio mais detalhadas durante o processo de ciência de dados que você conheceu.

Para o presente momento, vamos nos concentrar em poucos comandos para que você tenha noção de como a programação funciona e pode ser

estendida para outras situações mais complexas e reais do mercado. Assim, vamos aprender comandos da linguagem Python para:

1. Importar as bibliotecas necessárias ao projeto (Figura 3).

Figura 3 – importando bibliotecas

```
[ ] 1 # Bibliotecas para análise e processamento de dados
    2 import numpy as np
    3 import pandas as pd
    4 import matplotlib.pyplot as plt
    5 import pickle
```

```
[ ] 1 # Bibliotecas para pré-processamento e amostragem
    2 from sklearn.preprocessing import StandardScaler
    3 from sklearn.model_selection import train_test_split
```

```
[ ] 1 # Bibliotecas para aprendizagem e otimização
    2 from sklearn.neighbors import KNeighborsClassifier
    3 from sklearn.tree import DecisionTreeClassifier
    4 from sklearn.model_selection import GridSearchCV
```

Fonte: elaborada pelo autor.

2. Obter e observar os dados a partir de fontes externas, ou seja, que não são nativas das bibliotecas (Figura 4).

Figura 4 – Obtendo dados de fontes externas

```
[ ] 1 # Carregando os dados de uma planilha excel (xls) em um dataframe
    2 # Mais informações: https://biostat.app.vumc.org/wiki/Main/DataSets
    3 url = "https://biostat.app.vumc.org/wiki/pub/Main/DataSets/titanic3.xls"
    4 df = pd.read_excel(url)
```

```
[ ] 1 # Exibindo os dados carregados
    2 df.head()
```

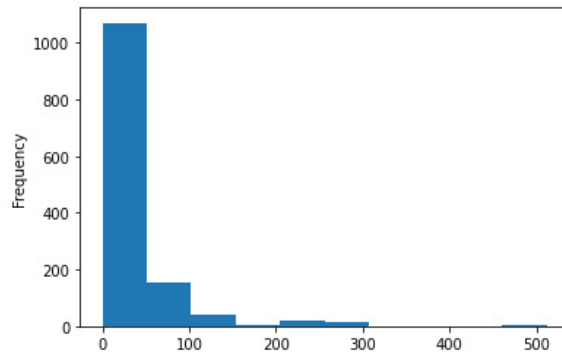
Fonte: elaborada pelo autor.

3. Visualizar os dados em formatos gráficos, observando uma ou mais variáveis em diferentes diagramas (Figura 5 e Figura 6).

Figura 5 – Visualização gráfica (histograma)

```
[ ] 1 # Plotando um histograma para o atributo 'fare' (tarifa)
    2 fig, ax = plt.subplots(figsize=(6, 4))
    3 df.fare.plot(kind="hist", ax=ax)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f5b92945358>

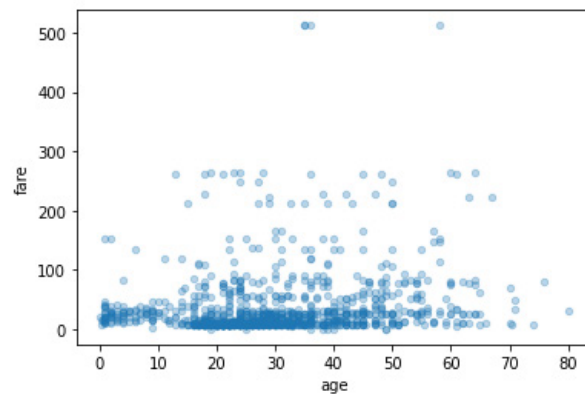


Fonte: elaborada pelo autor.

Figura 6 – Visualização gráfica (dispersão)

```
[ ] 1 # Plotando um gráfico de dispersão entre 'age' (idade) e 'fare' (tarifa)
    2 fig, ax = plt.subplots(figsize=(6, 4))
    3 df.plot.scatter(x="age", y="fare", ax=ax, alpha=0.3)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f5b923891d0>



Fonte: elaborada pelo autor.

4. Pré-processar alguns atributos antes de utilizá-los em algoritmos de aprendizagem de máquina (Figura 7).

Figura 7 – Atributos

```
[ ] 1 # Removendo atributos pouco ou nada significativos
2 df = df.drop(columns=["name",      # nome
3                               "ticket", # número da passagem
4                               "home.dest", # origem / destino
5                               "boat", # barco
6                               "body", # número de identificação do corpo
7                               "cabin"]) # número da cabine
8
9 # Criando variáveis dummy para atributos categóricos
10 df = pd.get_dummies(df, drop_first=True)
```

Fonte: elaborada pelo autor.

5. Separar os dados para treinamento e teste (Figura 8).

Figura 8 – Separação de dados

```
[ ] 1 # Separando os dados de entrada e saída
2 y = df.survived
3 X = df.drop(columns="survived")
4
5 # Separando aleatoriamente os dados para treinamento (70%) e teste (30%)
6 X_train, X_test, y_train, y_test = train_test_split(X,
7                                                    y,
8                                                    test_size=0.3,
9                                                    random_state=42)
10
11 # Exibindo as quantidades em cada subconjunto de dados
12 print("Treinamento: ", len(X_train))
13 print("Teste: ", len(X_test))
```

Treinamento: 916
Teste: 393

Fonte: elaborada pelo autor.

6. Construir e avaliar dois modelos preditivos para comparação (Figura 9).

Figura 9 – Construção de modelos preditivos

```
[ ] 1 knn = KNeighborsClassifier()
2 knn.fit(X_train, y_train)
3 knn.score(X_test, y_test)
```

0.7888040712468194

```
[ ] 1 tree = DecisionTreeClassifier()
2 tree.fit(X_train, y_train)
3 tree.score(X_test, y_test)
```

0.7048346055979644

Fonte: elaborada pelo autor.

7. Otimizar os hiperparâmetros do modelo com melhor desempenho da etapa anterior (Figura 10).

Figura 10 – Otimização do modelo

```
[ ] 1 # Otimizando os hiperparâmetros do algoritmo KNN
2 params = {'n_neighbors':list(range(1,11)),
3          'p':[1,2],
4          'weights':['uniform', 'distance'],
5          'algorithm':['auto', 'ball_tree','kd_tree','brute'],
6          'n_jobs':[-1]}
7
8 clf = GridSearchCV(estimator=knn, param_grid=params, cv=10)
9 best_model = clf.fit(X_train, y_train)
```

Fonte: elaborada pelo autor.

8. Distribuir o modelo em um formato que permita a sua reprodução sem novo processo de treinamento (Figura 11).

Figura 11 – Distribuição do modelo

```
[ ] 1 pickle.dump(best_model, open('modelo.sav', 'wb'))
```

Fonte: elaborada pelo autor.

Ao tentar executar os comandos acima, talvez você seja obrigado a realizar alguns ajustes para alcançar os mesmos resultados. Apesar disso, faz-se necessário observar os comandos para executar cada instrução desejada e o processo que construímos, do início ao fim, para a produção do modelo preditivo.



3. Desafios de análise e modelagem preditiva

Se todos os problemas fossem facilmente solucionáveis em poucas linhas de código, não teríamos centenas de livros sobre modelagem preditiva e projetos com longas durações, de 6 meses a 2 anos. O que estudamos até aqui é a base para todos os projetos, que poderão ser mais ou menos complexos, com maior ou menor volume de dados.

A fim de incrementar, vamos conhecer alguns dos principais desafios em análise e modelagem preditiva que você poderá enfrentar na sua jornada profissional.

3.1 Preparação de dados textuais

A maior parte dos problemas de modelagem preditiva trabalha com dados estruturados, que estão em uma organização e representação adequadas para análise de dados. Por outro lado, uma pequena e crescente parcela dos problemas tem exigido a adoção de dados não-estruturados, como os dados textuais.

Para utilizar bases de dados textuais em problemas de modelagem preditiva, precisaremos de técnicas adequadas para pré-processar os dados a fim de prepará-los adequadamente para os métodos de *Machine Learning*.

Em conjuntos de dados textuais (*corpora*), cada objeto é tratado como um documento, que pode assumir diferentes características no corpus, como tamanho do texto, tipo de conteúdo, idioma e tipo de linguagem. Nosso objetivo na preparação de dados textuais é a transformação de um corpus em um conjunto de dados capaz de descrever cada documento em termos de suas características (DA SILVA; PERES; BOSCARIOLI, 2017).

Uma das técnicas mais utilizada para representar documentos é conhecida como ***bag-of-words***, e consiste na construção de uma lista de ocorrências de palavras para formar um dicionário. Essa abordagem é muito útil para construir conjuntos de dados para descoberta de conhecimento.

Entre as aplicações mais comuns para dados textuais, estão o reconhecimento de SPAM em caixas de e-mails, a análise de sentimentos em redes sociais e a categorização de notícias de jornais.

3.2 Detecção de anomalias

A detecção de anomalias (*outliers*) é uma tarefa bastante requisitada em projetos de ciência de dados. Ela tem sido usada para detectar objetos anômalos da base de dados e, quando possível, executar alguma ação sobre eles. Neste ponto, uma pergunta importante a ser feita é: afinal, o que é uma anomalia?

Uma **anomalia** é um valor discrepante presente no conjunto de dados, ou seja, um valor que se localiza significativamente distante dos valores considerados normais. O objeto que apresenta esse valor discrepante parece desviar fortemente de outros membros da amostra à qual ele pertence, tornando-se inconsistente com o restante da base de dados. No contexto de domínio, as anomalias são padrões que não estão de acordo com uma noção bem definida de comportamento normal (DE CASTRO; FERRARI, 2016).

Na prática, a anomalia não pode ser compreendida por nós como um erro ou ruído nos dados, pois ela pode caracterizar um valor ou uma classe bem definida, mas de baixa ocorrência. Dessa forma, também não podemos ignorar a importância de detectar as anomalias na base de dados, já que uma fraude entre milhares de transações de cartão de crédito ou um intruso em uma centena de redes de computadores ainda podem ser prejudiciais.

Para detectar anomalias, vamos considerar esse problema como um problema de classificação binária, no caso de haver rótulos para os dados. Após definirmos o que será considerado como anomalia na aplicação, repetiremos todo o processo que já conhecemos em ciência de dados para buscar pelo modelo que discrimina entre os objetos normais e os objetos anômalos. Métricas como a acurácia, a sensibilidade (taxa de detecção) e a taxa de falsos positivos são comumente usadas nesse tipo de aplicação.

3.3 Desbalanceamento de classes

Alguns problemas são naturalmente desbalanceados. Por exemplo, as fraudes que acontecem em cartões de crédito são eventos raros frente às milhões de transações diárias em um sistema de meio de pagamentos. Se a quantidade de transações rejeitadas fosse proporcional à quantidade de transações aceitas, certamente o pagamento por cartão de crédito seria proibido e muitas pessoas deixaram de confiar nessa forma de pagamento.

O uso de técnicas preditivas para detectar fraudes, também vai exigir que os dados sejam pré-processados. Caso o tratamento não seja realizado, os algoritmos poderão favorecer a classificação de novos dados na classe majoritária, ou seja, as novas transações serão todas classificadas como transações comuns, ainda que sejam fraudes verdadeiras.

Para fins experimentais, precisaremos balancear as classes sem interferir na natureza do problema. A melhor estratégia é gerar novos dados a partir do processo original e equilibrar as classes. Porém, na maior parte das vezes, isso não é possível, e uma alternativa é balancear os dados de maneira artificial (FACELLI et al., 2011), usando uma das seguintes estratégias:

- **Redefinir o tamanho do conjunto de dados:** nesta situação, podemos tanto incluir novos objetos à classe minoritária quanto eliminar objetos da classe majoritária.
- **Utilizar diferentes custos de classificação:** podemos fazer com que os poucos objetos da classe minoritária tenham o mesmo peso dos muitos objetos da classe majoritária por meio de custos diferentes que tenham como efeito privilegiar a classificação na classe minoritária.

- **Induzir um modelo para uma classe:** nesta situação, as classes são aprendidas separadamente pelo algoritmo. Embora menos comum, a classificação com apenas uma classe pode ser uma alternativa interessante no tratamento de dados desbalanceados.

Essas estratégias para o tratamento de dados desbalanceados colaboram para o desempenho preditivo do problema, mas ainda exigem que as outras etapas do processo sejam complementadas por estratégias próprias, como abordagens para os algoritmos e métricas específicas de avaliação de desempenho (FERNÁNDEZ et al., 2018).

Nesta unidade, você se concentrou em alguns temas importantes da análise e modelagem preditiva para o seu aprendizado, tais como:

- O processo de desenvolvimento de projetos de ciência de dados.
- A linguagem de programação Python aplicada à análise e dados.
- Mecanismos de programação para modelagem e entrega de modelos e resultados.
- Soluções mais comuns para resolver problemas de modelagem.
- Os principais desafios que podemos encontrar na análise e modelagem preditiva.



Referências

CHAPMAN, Pete et al. CRISP-DM 1.0: Step-by-step data mining guide. **SPSS inc**, v. 9, p. 13, 2000. Disponível em: <https://the-modeling-agency.com/crisp-dm.pdf>. Acesso em: 12 maio 2021.

DA SILVA, Leandro A.; PERES, Sarajane M.; BOSCARIOLI, Clodis. **Introdução à mineração de dados:** com aplicações em R. São Paulo: Elsevier Brasil, 2017.

DE CASTRO, Leandro N.; FERRARI, Daniel G. **Introdução à mineração de dados:** conceitos básicos, algoritmos e aplicações. São Paulo: Saraiva, 2016.

FACELI, Katti et al. **Inteligência Artificial:** uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2011.

FERNÁNDEZ, Alberto et al. **Learning from imbalanced data sets.** Berlin: Springer, 2018.

MCKINNEY, Wes. **Python para análise de dados:** tratamento de dados com Pandas, NumPy e IPython. São Paulo: Novatec Editora, 2019.

WAZLAWICK, Raul Sidnei. **Introdução a Algoritmos e Programação com Python:** Uma abordagem dirigida por testes. Rio de Janeiro: Elsevier, 2018.



BONS ESTUDOS!